



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 4: Persistencia Avanzada y Optimización de Rendimiento con Spring Data JPA e Hibernate

Resumen

Esta práctica guiada tiene como objetivo instruir al estudiantado en técnicas avanzadas de persistencia objeto-relacional (ORM) e Hibernate en Spring Boot. El laboratorio se divide en dos secciones principales. En la primera sección, usted implementará la auditoría automática de entidades mediante superclases JPA y diseñará relaciones bidireccionales One-to-Many controlando la persistencia en cascada.

En la segunda sección, usted explorará el funcionamiento interno de Hibernate mediante transacciones del ciclo de vida de los objetos en memoria utilizando el EntityManager. Además, diagnosticará y resolverá el clásico problema de rendimiento **N+1 SELECT** analizando las trazas de

Metadatos del Laboratorio

- **Tiempo estimado:** 4 horas.
- **Herramientas requeridas:** Java (versión local instalada), SQL Server Developer Edition, SQL Server Management Studio (SSMS), Maven.

Metas de Aprendizaje:

1. Estructurar una superclase de auditoría `@MappedSuperclass` para registrar automáticamente fechas de creación y modificación de registros.
2. Modelar y mapear una relación bidireccional Many-to-One y One-to-Many controlando la integridad física mediante `orphanRemoval`.
3. Monitorear los estados transitorios, persistentes y desprendidos de los objetos JPA utilizando el `EntityManager` de Hibernate.
4. Diagnosticar el fallo de rendimiento N+1 SELECT y solucionarlo mediante consultas preparadas con JOIN FETCH.

Conceptos Clave (El 'Qué')

1. Auditoría Automática con JPA Auditing

En sistemas empresariales, cada tabla debe registrar metadatos de control: la fecha en que se insertó una fila y la fecha de su última modificación. Spring Data JPA automatiza esto capturando los eventos del ciclo de vida antes de persistir (`@PrePersist`) o actualizar (`@PreUpdate`).

2. Estados del Ciclo de Vida de Entidades

Hibernate administra en memoria RAM el ciclo de vida de los objetos mediante tres estados principales: * **Transient (Transitorio):** Un objeto nuevo creado en Java que no existe en base de

datos. * **Persistent (Persistente / Managed)**: Un objeto asociado al contexto de persistencia. Cualquier seteo de propiedades se guardará en base de datos de manera automática mediante *Dirty Checking* al finalizar la transacción. * **Detached (Desprendido)**: Un objeto con identificador en base de datos pero cuya sesión de persistencia ha finalizado, por lo que los cambios realizados en memoria no se guardan solos.

3. El Problema N+1 SELECT

Ocurre cuando la carga perezosa (`FetchType.LAZY`) se ejecuta iterativamente dentro de una lista de registros. Hibernate realiza 1 consulta inicial para cargar la lista de entidades padre y luego ejecuta N consultas secundarias adicionales para traer los registros hijos asociados a cada uno de los padres, mermando el rendimiento del servidor.

Guía de Base de Datos (SSMS Script)

Para esta práctica guiada, continuará utilizando la base de datos `VideoRent[Carné]_II2026` creada en la Práctica Guiada 3. Debe realizar las siguientes modificaciones para habilitar las columnas de auditoría y la tabla de reseñas.

1. Abra SQL Server Management Studio (SSMS) y conéctese a su motor local.
2. Ejecute el siguiente script en una nueva consulta para actualizar su base de datos (sustituya `[Carné]` por su carnet universitario en mayúsculas):

```
USE VideoRent[Carné]_II2026;
GO

-- 1. Agregar columnas de auditoria a la tabla Pelicula
ALTER TABLE Pelicula ADD
    fecha_creacion DATETIME NULL,
    fecha_modificacion DATETIME NULL;
GO

-- 2. Crear tabla Review para el mapeo bidireccional
CREATE TABLE Review (
    review_id INT IDENTITY(1,1) PRIMARY KEY,
    comentario VARCHAR(500) NOT NULL,
    calificacion INT NOT NULL,
    pelicula_id INT NOT NULL,
```

```

    fecha_creacion DATETIME NULL,
    fecha_modificacion DATETIME NULL,
    -- FK entre Review / Pelicula
    CONSTRAINT FK_Review_Pelicula FOREIGN KEY (pelicula_id)
        REFERENCES Pelicula(pelicula_id) ON DELETE CASCADE
);
GO

-- 3. Insertar semillas de reseñas para pruebas de rendimiento
INSERT INTO Review (comentario, calificacion, pelicula_id,
                    fecha_creacion, fecha_modificacion)
VALUES
('Excelente pelicula, muy recomendada.', 5, 1, GETDATE(), GETDATE()),
('Los efectos especiales son buenos.', 3, 1, GETDATE(), GETDATE()),
('Una obra maestra del genero cyberpunk.', 5, 1, GETDATE(), GETDATE());
GO

```

Sección 1: Spring Data JPA Avanzado (Auditoría y Relaciones)

Reemplace la sección `<carnet>` por su carnet universitario en minúsculas en las declaraciones de paquetes de Java.

1. Configuración del Archivo pom.xml (Maven):

Para trabajar con persistencia avanzada, verifique que su archivo `pom.xml` incluya la dependencia del starter de datos JPA. Esta dependencia incluye tanto la API de Spring Data JPA como la implementación del motor ORM Hibernate:

```

<!-- Spring Data JPA e Hibernate -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

```

2. Crear la Entidad Base de Auditoría (AuditableEntity.java):

Cree esta clase abstracta base en el paquete `domain`. Sus propiedades serán heredadas por el resto de entidades físicas de su aplicación:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/AuditableEntity.java`

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.domain;

import jakarta.persistence.Column;
import jakarta.persistence.EntityListeners;
import jakarta.persistence.MappedSuperclass;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.annotation.LastModifiedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;
import java.time.LocalDateTime;

@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class AuditableEntity {

    @CreatedDate
    @Column(name = "fecha_creacion", updatable = false)
    private LocalDateTime fechaCreacion;

    @LastModifiedDate
    @Column(name = "fecha_modificacion")
    private LocalDateTime fechaModificacion;

    public LocalDateTime getFechaCreacion() { return fechaCreacion; }
    public void setFechaCreacion(LocalDateTime f) { this.fechaCreacion = f; }

    public LocalDateTime getFechaModificacion() { return fechaModificacion; }
    public void setFechaModificacion(LocalDateTime f) {
        this.fechaModificacion = f;
    }
}

```

3. Modificar la Entidad Pelicula.java:

Actualice la entidad Pelicula.java para que extienda de AuditableEntity y defina la relación bidireccional de un muchos a uno (@OneToMany) hacia Review:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/Pelicula.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.domain;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "Pelicula")

```

```

public class Pelicula extends AuditableEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "pelicula_id")
    private Integer id;

    @Column(nullable = false, length = 50)
    private String titulo;

    @Column(nullable = false)
    private boolean subtitulada;

    @Column(nullable = false)
    private boolean estreno;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "genero_id", nullable = false)
    private Genero genero;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "PeliculaActor",
        joinColumns = @JoinColumn(name = "pelicula_id"),
        inverseJoinColumns = @JoinColumn(name = "actor_id")
    )
    private List<Actor> actores = new ArrayList<>();

    // Relacion bidireccional mapeada por el atributo en Review
    @OneToMany(
        mappedBy = "pelicula",
        cascade = CascadeType.ALL,
        orphanRemoval = true,
        fetch = FetchType.LAZY
    )
    private List<Review> reviews = new ArrayList<>();

    public Pelicula() {}

    // Getters y Setters...
    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getTitulo() { return titulo; }
    public void setTitulo(String t) { this.titulo = t; }
}

```

```

    public boolean isSubtitulada() { return subtitulada; }
    public void setSubtitulada(boolean s) { this.subtitulada = s; }

    public boolean isEstreno() { return estreno; }
    public void setEstreno(boolean e) { this.estreno = e; }

    public Genero getGenero() { return genero; }
    public void setGenero(Genero g) { this.genero = g; }

    public List<Actor> getActores() { return actores; }
    public void setActores(List<Actor> a) { this.actores = a; }

    public List<Review> getReviews() { return reviews; }
    public void setReviews(List<Review> r) { this.reviews = r; }

    // Helpers bidireccionales en memoria
    public void addReview(Review review) {
        reviews.add(review);
        review.setPelicula(this);
    }

    public void removeReview(Review review) {
        reviews.remove(review);
        review.setPelicula(null);
    }
}

```

4. Crear la Entidad Review.java:

Defina la clase hija de persistencia mapeada hacia la base de datos SQL Server:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/domain/Review.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.domain;

import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;

@Entity
@Table(name = "Review")
public class Review extends AuditableEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "review_id")
    private Integer id;
}

```

```

    @Column(nullable = false, length = 500)
    private String comentario;

    @Column(nullable = false)
    private int calificacion;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "pelicula_id", nullable = false)
    @JsonIgnore // Evita ciclos infinitos al serializar JSON
    private Pelicula pelicula;

    public Review() {}

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getComentario() { return comentario; }
    public void setComentario(String c) { this.comentario = c; }

    public int getCalificacion() { return calificacion; }
    public void setCalificacion(int c) { this.calificacion = c; }

    public Pelicula getPelicula() { return pelicula; }
    public void setPelicula(Pelicula p) { this.pelicula = p; }
}

```

5. Habilitar Auditoría JPA (JpaConfig.java):

Cree una clase de configuración en el paquete config para habilitar el interceptor automático de Spring Data JPA:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/config/JpaConfig.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica3.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.data.jpa.repository.config.EnableJpaAuditing;

@Configuration
@EnableJpaAuditing
public class JpaConfig {
}

```

6. Crear ReviewRepository.java:

Cree el repositorio JPA para las reseñas en el paquete data:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/data/ReviewRepository.java

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.data;

import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Review;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface ReviewRepository
    extends JpaRepository<Review, Integer> {
}
```

7. Verificación Práctica de Auditoría:

Ejecute la aplicación mediante consola (mvn spring-boot:run), inserte una película y reseña utilizando Swagger UI y consulte SSMS. Las columnas fecha_creacion y fecha_modificacion en ambas tablas deben mostrar la hora exacta de la transacción de forma automatizada.

Sección 2: Hibernate Internals (Ciclo de Vida y N+1 Select)

A continuación, crearemos servicios específicos para depurar y optimizar el motor ORM Hibernate.

1. Servicio de Ciclo de Vida y Dirty Checking (DemoHibernateService.java):

Cree un servicio en el paquete business que utilice el EntityManager inyectado para probar transiciones de estado:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/business/DemoHibernateService.java

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.business;

import cr.ac.ucr.paraiso.ie.carnet.practica3.data.PeliculaRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Pelicula;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
```

```

public class DemoHibernateService {

    @PersistenceContext
    private EntityManager entityManager;

    private final PeliculaRepository repository;

    public DemoHibernateService(PeliculaRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public void probarCicloVidaYDirtyChecking(Integer id) {
        // A. ESTADO TRANSITORIO (Transient)
        Pelicula nueva = new Pelicula();
        nueva.setTitulo("Inception");

        // B. ESTADO PERSISTENTE (Managed)
        // El EntityManager la empieza a rastrear
        entityManager.persist(nueva);

        // C. DIRTY CHECKING (Guardado automatico)
        Pelicula existente = repository.findById(id)
            .orElseThrow(() -> new IllegalArgumentException("No existe"));

        // Modificación del objeto persistente
        existente.setTitulo("The Matrix Modificada");

        // Al terminar el método transaccional, Hibernate detectará el cambio
        // y enviará el SQL UPDATE a base de datos de forma automática.
    }

    @Transactional
    public void probarEstadoDetached(Integer id) {
        Pelicula p = repository.findById(id).orElseThrow();

        // Desasociar del contexto de persistencia
        entityManager.detach(p); // Estado DETACHED

        p.setTitulo("Titulo en Memoria");

        // Al terminar la transacción, el cambio de título NO se guardará
        // ya que la entidad ya no está administrada en la Caché L1.
    }
}

```

2. Taller de Diagnóstico de N+1 SELECT:

Habilite el logueo de sentencias SQL en el archivo `application.properties`:

```
spring.jpa.properties.hibernate.show_sql=true
```

Agregue un método en el controlador `PeliculaController.java` para comprobar cómo el acceso secuencial en la colección perezosa (LAZY) lanza consultas reiterativas en bucle:

```
@GetMapping("/nplusone")
@Operation(summary = "Demostrar problema N+1 SELECT")
public List<String> testNPlusOne() {
    // 1 Consulta inicial para traer N películas
    List<Película> películas = repository.findAll();
    List<String> list = new ArrayList<>();
    for (Película p : películas) {
        // N consultas adicionales para cargar la relación perezosa
        for (Review r : p.getReviews()) {
            list.add(p.getTitulo() + ": " + r.getComentario());
        }
    }
    return list;
}
```

Ejecute la petición en Swagger. Verifique la terminal de su aplicación: observará múltiples sentencias `SELECT` consecutivas para la misma relación.

3. Optimización del Rendimiento con JOIN FETCH:

Actualice su repositorio `PeliculaRepository.java` declarando una consulta optimizada que traiga la información en un solo viaje:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica3/data/PeliculaRepository.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica3.data;

import cr.ac.ucr.paraiso.ie.carnet.practica3.domain.Película;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface PeliculaRepository
```

```

    extends JpaRepository<Pelicula, Integer> {
    // JOIN FETCH: Resuelve N+1 en una sola consulta de base de datos
    @Query("SELECT DISTINCT p FROM Pelicula p LEFT JOIN FETCH p.reviews")
    List<Pelicula> obtenerPeliculasConReviewsOptimizadas();
}

```

Agregue el endpoint optimizado en su `PeliculaController.java`:

```

    @GetMapping("/optimized")
    @Operation(summary = "Resolver N+1 SELECT con JOIN FETCH")
    public List<String> testOptimized() {
        // Ejecuta 1 unica consulta optimizada
        List<Pelicula> peliculas = repository
            .obtenerPeliculasConReviewsOptimizadas();
        List<String> list = new ArrayList<>();
        for (Pelicula p : peliculas) {
            for (Review r : p.getReviews()) {
                list.add(p.getTitulo() + ": " + r.getComentario());
            }
        }
        return list;
    }
}

```

Ejecute este nuevo endpoint y compruebe los logs. Encontrará que Hibernate envió **una única consulta SELECT con un LEFT OUTER JOIN** al motor SQL Server, optimizando el rendimiento.

Taller de Depuración (El Error Común)

El objetivo de este taller es diagnosticar y reparar la excepción `LazyInitializationException`.

Paso 1: Provocar el Error

- Modifique temporalmente su endpoint `GET /api/peliculas/{id}` en el controlador para devolver la lista de reseñas de la película obtenida:

```
@GetMapping("/{id}/reviews-error")
public ResponseEntity<?> getReviewsError(@PathVariable Integer id) {
    Pelicula p = repository.findById(id).orElseThrow();
    // Intento de acceder a la colección LAZY fuera de la transacción
    return ResponseEntity.ok(p.getReviews());
}
```

- Al consultar el endpoint mediante Swagger UI, usted recibirá una traza de excepción similar a la siguiente: `org.hibernate.LazyInitializationException: could not initialize proxy - no Session`

Paso 2: Diagnóstico

El error ocurre porque la entidad `Pelicula` entra en estado **Detached** al finalizar el método del repositorio `findById` (se cierra la sesión de persistencia). Al intentar leer la colección perezosa `getReviews()` en el controlador REST, ya no existe una transacción abierta que provea conexión a SQL Server para cargar los datos.

Paso 3: Resolución del Error

La forma más limpia y óptima de resolver este error sin cargar todos los datos de forma ansiosa (EAGER de forma permanente) consiste en diseñar una consulta que precargue la relación requerida únicamente cuando sea necesario. En su repositorio de películas, implemente:

```
@Query("SELECT p FROM Pelicula p LEFT JOIN FETCH p.reviews WHERE p.id = :id")
Optional<Pelicula> findByIdConReviews(@Param("id") Integer id);
```

Modifique el controlador para recuperar la película con sus reseñas cargadas de forma anticipada. Pruebe de nuevo el endpoint y valide que retorne la información sin fallos.

Parte 3: Reto Autónomo (Sin Solución)

Debe implementar de forma autónoma el borrado o archivado masivo de reseñas basado en bajas calificaciones.

Requerimientos:

- En `ReviewRepository.java`, cree un método anotado con `@Query` y `@Modifying` para archivar los comentarios de las reseñas cuya calificación sea menor o igual al parámetro recibido.
- La modificación en la base de datos debe concatenar el texto "[ARCHIVADA] " al inicio del comentario de la reseña.
- **Importante:** Asegure la consistencia de la Caché L1 configurando la limpieza automática en la anotación: `@Modifying(clearAutomatically = true)`.
- Exponga el endpoint a través de una petición REST de tipo PUT/PATCH y compruebe que los registros en la base de datos cambian adecuadamente mediante sentencias SELECT en SQL Server Management Studio (SSMS).